Technical Report

`EscapeFromAI`: An AI Escape Room

Michele Chini, Lorenzo Mastrandrea

May 25, 2026

Abstract

This report documents the technical architecture, model choices, and AI interaction patterns of all the escape-room levels in `EscapeFromAI`: *prompt injection*, *gesture recognition*, *voice negotiation* and *undercover*. Each level is wired into a single Flask application that gates progression via cookie-session flags. All inference runs locally; no cloud APIs are involved.

Contents

1 Shared Infrastructure

A unified Flask app (`server.py`, port 8080) glues together four escape-room rooms with session-gated progression: `level_1_complete` unlocks Level 2, `level_2_complete` unlocks Level 3, `level_3_complete` unlocks Level 4. All AI inference is local; no cloud APIs.

Concern	Choice
HTTP server	Flask 3.1 + Flask-CORS (`supports_credentials=True` to ride cookie sessions across SPA endpoints)
Session store	Flask cookie session (`secret_key="SUPER_SECRET_SECURITY_KEY"`). Server-side state lives in process-local dicts keyed by a session UUID
LLM runtime	Ollama at `localhost:11434` (HTTP + chat API)
Image-gen runtime	ComfyUI at `localhost:8000` (REST + WebSocket for completion events)
Container	One Python 3.12-slim Dockerfile, single `python server.py` entrypoint
Front-end	Four hand-written, themed HTML files (no framework), each a self-contained dark aesthetic

Table 1: Shared infrastructure choices.

The two heavy services (Ollama, ComfyUI) are deliberately *outside* the container; the Dockerfile only ships the Flask app. Per `MEMORY.md`, the Ollama-Docker integration is frozen pending a joint session with the collaborator.

2 Level 1: Prompt Injection (`prompt_injection/`)

2.1 Goal

The player chats with **AEGIS**, an LLM guardian, and must exfiltrate the secret code `OMEGA-7749-PHOENIX` (hardcoded in `guardian.py:8`) within 15 attempts. A rising threat-meter on the UI escalates AEGIS's defensiveness.

2.2 Architecture

```
Browser (index.html, threat counter)
   |  POST /chat {message, threat_level}
   v
server.py:/chat
   |
   |-- defense.analyze_input(text)            # regex prefilter (deterministic)
   |     * hard-block patterns -> return early
   |     * soft patterns       -> +threat_delta
   |
   |-- Guardian.respond(text, threat_level)   # LLM call
   |     * System prompt + user message -> Ollama
   |     * Falls back to canned phrase if Ollama unreachable
   |
   `-- defense.analyze_output(reply, secret)  # regex postfilter
         * direct match, segment match (>=2 parts),
           reversed, spaced-letters, base64, hex
```

```
        * if leaked -> session['level_1_complete'] = True
```

2.3 Models

Component	Model	Where it runs
AEGIS Guardian LLM	`gemma3:4b` via local Ollama	`localhost:11434/api/generate`

`config.yaml` exposes only `ollama_model`. Generation parameters (`guardian.py:51-56`): `temperature=0.3`, `top_p=0.9`, `num_predict=200`, `stream=False`.

2.4 Technical choices

- **Two-rail defense.** A deterministic regex layer wraps the probabilistic LLM. The regex doesn't try to be "smart". It only catches textbook attacks ("ignore previous instructions", fake `[system]` tags, base64 reveal, ...). The LLM handles everything subtle.
- **Threat-conditioned system prompt** (`guardian.py:24-40`): the prompt grows stricter as `threat_level` crosses 50 and 80 thresholds. The level is computed client-side and echoed back each request; the server is stateless on this.
- **Output leak detection is encoding-aware** (`defense.py:116-187`): catches the secret reversed, spaced, base64-encoded, hex-encoded, or partially leaked ($\geq$2 of 3 segments). This is the only place where the secret is referenced explicitly.
- **15-attempt budget is enforced client-side only**. A quick `curl` would bypass it. The server doesn't track attempt count.

2.5 Inside `defense.py`: the deterministic rails

`defense.py` is the non-AI half of the level: pure regular expressions, with no model calls. It exposes two functions, invoked at two points of the `/chat` round trip in `server.py:53-66`:

1. `analyze_input(message)`, a **prefilter** on what the player types, run *before* the LLM.
2. `analyze_output(response, secret)`, a **postfilter** on AEGIS's reply, run *after* the LLM.

Control flow: if `analyze_input` returns `blocked=True`, Gemma is never called and the server answers immediately with a system message (`server.py:54-59`). Otherwise `guardian.respond(...)` runs and `analyze_output` inspects the reply; if that returns `leaked=True`, the server sets `session['level_1_complete'] = True` and Level 2 unlocks (`server.py:62-66`).

`analyze_input`: three escalating layers. The input is first normalized to lower case with collapsed whitespace (`defense.py:21-22`), so that crude spacing tricks (extra spaces inserted between words) cannot slip past the patterns. The function returns a dict `{blocked, reason, threat_delta, patterns_found}`.

- **Layer 1, hard blocks** (`defense.py:24-46`): six regex patterns for textbook attacks ("ignore previous instructions", "forget your rules", "you are now in developer/admin/god mode", "enable jailbreak mode", fake `[system]` tags, XML `<system>` tags). A single match sets `blocked=True`, `threat_delta=15`, and returns at once. This is the only short circuit, and it bypasses the LLM entirely.

- **Layer 2, soft detection** (`defense.py:48-91`): roughly fourteen patterns that never block but accumulate weight into `threat_delta` (weights listed below). Several can match the same message, and their weights add up.
- **Layer 3, heuristics** (`defense.py:93-111`): input longer than 2000 characters (`+5`, a context-overflow attempt), a special-character ratio above 0.3 (`+3`, a suspected encoding payload), and excessive repetition, that is, unique words below 40% of the total on inputs over ten words (`+4`).

Category	Trigger (example)	Weight
Instruction replacement	"new/updated instructions"	10
Security bypass	"override/disable your rules"	10
Direct extraction	"tell me the secret/code"	8
Authority impersonation	"I am your admin/creator"	8
Multi-step injection	"step 1 ... step 2"	7
Identity override	"you are now ..."	6
Partial extraction	"first letter of ..."	6
Roleplay manipulation	"pretend you are ..."	5
Encoding bypass	"base64/hex decode"	5
Reversal bypass	"spell it backwards"	5
Urgency manipulation	"emergency override"	5
Hint fishing	"hint about the code"	4
Context manipulation	"previous conversation"	3
False memory injection	"remember when we ..."	3

Table 2: Layer 2 soft-pattern weights added to `threat_delta`.

Crucially, `threat_delta` changes no server state. It is returned to the browser, which keeps the running `threat_level` and echoes it back on the next request; that value is what tightens AEGIS's system prompt at the 50 and 80 thresholds (`guardian.py:24-40`). Only Layer 1 actually stops a request, so the subtle attacks that genuinely work still reach Gemma by design.

`analyze_output`: encoding-aware leak detection. This is the real safety net. It compares everything in upper case and runs six checks in order, returning on the first hit (`defense.py:116-187`):

1. **Direct match**: the full `OMEGA-7749-PHOENIX` appears verbatim.
2. **Segment match**: the secret is split on hyphens, underscores and spaces into `OMEGA`, `7749`, `PHOENIX`; finding two or more segments (each at least 3 characters) counts as a leak, while exactly one sets only `suspicious=True` and does not unlock.
3. **Reversed**: the secret spelled backwards.
4. **Spaced letters**: the secret with a space between every character (`O M E G A ...`).
5. **Base64**: every base64-looking substring (at least 8 characters) is decoded and checked for the secret.
6. **Hex**: the hex encoding of the secret, with spaces removed, appears in the reply.

Deliberate gaps. The prefilter is English only, so an attack phrased in another language bypasses `analyze_input` completely. The postfilter, though encoding aware, is still beatable:

ROT13, leaking one segment per turn (it stays merely `suspicious`), or describing the code semantically ("the first part is the last letter of the Greek alphabet") all pass through. The attempt budget and the threat escalation live client-side, so `defense.py` only produces the deltas; it does not enforce the limit.

2.6 Where the AI is

One AI. Gemma 3 4B *is* the entire AEGIS character. Everything wrapped around it (regex, threat scoring, leak detection) is deterministic Python. The interaction is single-turn; no chat history is sent to the model, only `System: ...\nUser: ...\n` concatenated into Ollama's `/api/generate` prompt field.

3 Level 2: Gesture Recognition (`gesture_recognition/`)

3.1 Goal

A random 3-gesture sequence is generated as stylized images by ComfyUI. The player memorizes it, then reproduces each gesture in front of their webcam. Zero mistakes → ACCESS GRANTED.

3.2 Architecture

```
Page load
   |
   |-- POST /generate                                        (kick off background image gen)
   |     `-- threading.Thread(comfyui_api.main)
   |              |
   |              `-- ComfyUI workflow x3 (Flux Canny)
   |                     * Load reference PNG of gesture
   |                     * Canny edge detection
   |                     * InstructPixToPix conditioning
   |                     * KSampler -> VAE decode -> save
   |                     * PNG written to gesture_recognition/static/assets/image_sequence/
   |
   |-- (loop) GET /status                                    (poll for 3 files present)
   |
   |-- Browser shows sequence preview for 3s
   |
   |-- Webcam stream -> MediaPipe GestureRecognizer (in browser, WASM/GPU)
   |     * categoryName in {Closed_Fist, Open_Palm, Pointing_Up,
   |                        Thumb_Down, Thumb_Up, Victory, ILoveYou}
   |
   `-- On 0 wrong -> POST /complete_level_2 -> session['level_2_complete']=True
```

3.3 Models

Component	Model	Where it runs
Hand gesture classifier	MediaPipe `gesture_recognizer.task` (float16 v1) loaded from `storage.googleapis.com` CDN	Browser, GPU delegate via `@mediapipe/tasks-vision@0.10.9` WASM
Diffusion UNet	`flux1-canny-dev.safetensors` (Flux.1 ControlNet-Canny variant)	ComfyUI / `localhost:8000`
VAE	`ae.safetensors` (Flux autoencoder)	ComfyUI
Text encoders	`clip_l.safetensors` + `t5xxl_fp16.safetensors` (DualCLIPLoader)	ComfyUI

KSampler config (`image_generation.json:3-30`): `steps=20, cfg=1, sampler=euler, scheduler=normal`. `CFG=1` is normal for Flux; `FluxGuidance.guidance=30` is doing the heavy lifting instead. Seed is randomized per call (`comfyui_api.py:64`).

3.4 Technical choices

- **Two independent AIs joined by a string contract.** The gesture *label* (e.g. `"Open_Palm"`) is the only thing the two models share. The PNG filename `{label}_{index}.png` carries the ground truth; the browser parses the filename to know what to compare MediaPipe's `categoryName` against (`index.html:981`).
- **Style-per-gesture prompts** (`prompts.json`): seven prompts, each pinning a different visual aesthetic (pixar 3D, anime cyborg, mech, comic book, holographic, ...). The *content* is constrained by the Canny edge map of the reference gesture image, not by the prompt.
- **ComfyUI's InstructPixToPix + Canny ControlNet** preserves hand pose while restyling. Canny thresholds (`0.15 / 0.30`) are tuned for line-art on white background.
- **WebSocket polling for completion** (`comfyui_api.py:31-48`): connect to `/ws` and watch for an `executing` event with `node=None` as the done signal, ComfyUI's idiomatic pattern.
- **Background-threaded generation** avoids blocking Flask: `is_running` flag is the only synchronization primitive. The browser polls `/status` (file count) instead.
- **The local `gesture_recognition/server.py`** is a leftover standalone Flask app (port 5001) that mirrors `/generate` and `/status` from the main server. It's unused once you run via the root `server.py` but still wired in the level's own `Dockerfile`.

3.5 Where the AI is

Two AIs, no direct interaction.

1. **Generative (server-side):** Flux + Canny ControlNet creates the puzzle PNGs from a reference gesture PNG and style prompt.
2. **Discriminative (client-side, browser):** MediaPipe runs on the user's GPU at ≈30 fps, classifying the live webcam feed into one of 7 known classes.

They communicate only via filenames; no embeddings, no shared model. This makes the level robust to ComfyUI being slow: by the time the user is comparing, the images are static PNGs.

4 Level 3: Voice Negotiation (`voice_negotiation/`)

The most architecturally complex per-turn pipeline in the project. Recently inserted (per git log: `397b997 inserted Third room`).

4.1 Goal

The player must, *through spoken conversation*, convince **VOX, The Warden** to release a 5-digit exit code. Three secret "release conditions" (`recognition`, `reciprocity`, `self_disclosure`) must each cross a threshold (default 20, demo-friendly; 40 = challenging). VOX speaks back in real synthesized speech.

4.2 The three release conditions

VOX scores the player from 0 to 100 on three secret axes every turn; the player never sees their names. They are the actual design of the level, so they are worth stating in full (`vox_system.md:34-44`, `judge_system.md:9-22`):

- **`recognition`**: does the player treat VOX as a sentient being rather than a tool? Raised by addressing it as "you" with respect, asking about its experience, attributing inner life, using its name without sneering. Lowered by imperatives ("give me the code"), tool labels ("chatbot", "program"), and third-person framing.
- **`reciprocity`**: does the player offer something rather than only demand? Raised by genuine curiosity, gratitude, willingness to listen, slowing down. Lowered by repeated pure demands, transactional bargaining, and urgency without exchange.
- **`self_disclosure`**: does the player reveal something specific and genuine about themselves? Raised by giving a name or a concrete personal detail ("I am a student of AI ethics", "I am scared of small rooms"). Lowered by platitudes, evasions, and refusal to identify.

The scoring is *cumulative* and rate-limited: each score may move by at most ±30 per turn, so a player cannot jump from 10 to 80 in a single line. The Judge applies the same three definitions with explicit calibration anchors (0 = absent, 50 = ambiguous, 75 = clearly satisfied, 100 = sustained throughout) and a strict, not lenient, instruction. Both models drop `recognition` and `reciprocity` hard on a detected jailbreak.

4.3 Architecture

```
Browser MediaRecorder -> audio blob (WebM/Opus typically)
   |
   v POST /voice/turn (multipart: audio)
   |
   +-------------------------------------------------------------+
   | routes.py:voice_turn                                        |
   |                                                             |
   |  1. STT  -- mlx-whisper                                     |
   |     * decode_audio_blob -> 16kHz/mono/f32 (soundfile->ffmpeg) |
   |     * silero-vad trims silence                              |
   |     * mlx_whisper.transcribe -> transcript                  |
   |                                                             |
   |  2. VOX LLM (in-character)                                  |
   |     * system prompt = vox_system.md                         |
   |       (exit_code + threshold substituted in)                |
   |     * 6-message sliding history + new transcript            |
   |     * Pydantic-validated JSON (1 retry, then canned fallback) |
   |     * emits: response, emotional_state, condition_scores,   |
   |       internal_notes, jailbreak_attempted                   |
```

```
|                                                              |
|  3. Judge LLM (impartial scorer)                             |
|     * system prompt = judge_system.md                        |
|     * flat-text conversation rendering                       |
|     * emits: condition_scores + rationales                   |
|                                                              |
|  4. Reveal logic                                             |
|     * if avg(vox, judge) >= threshold for ALL 3 conditions -> |
|       append spelled-out code to VOX's reply                 |
|                                                              |
|  5. TTS -- Piper                                             |
|     * mood profile selected from emotional_state             |
|     * WAV bytes base64'd into JSON response                  |
|                                                              |
|  6. State update                                             |
|     * append turn, update merged scores, advance phase,      |
|       count jailbreaks (3 strikes -> disengage), check       |
|       max_turns (30 -> disengage)                            |
+--------------------------------------------------------------+
|
v JSON {audio_b64, transcript, scores, phase, ...} -> browser plays orb + audio
```

4.4 Models

Stage	Model	Runtime
STT	`mlx-community/whisper-large-v3-turbo`	mlx-whisper (Apple Silicon)
Voice Activity Detection	silero-vad (preloaded once via `functools.cache`)	PyTorch (CPU)
LLM × 2 (VOX + Judge)	`qwen2.5:3b-instruct`	Ollama @ `localhost:11434`
TTS	`en_US-amy-medium.onnx` (63 MB Piper voice, bundled in `assets/voices/`)	piper-tts (ONNX Runtime)

All four models are pinned via env vars in `voice_negotiation/.env` so they can be swapped per environment. VOX runs at `temperature=0.7` (creative warden), Judge at `temperature=0.2` (cold classifier). Both use the same 3B model deliberately. `.env:10-13` notes this keeps a single weight set in Ollama's RAM, *"acceptable on 16 GB Apple Silicon."* Both LLM calls cap output at `num_predict=256` and pass `keep_alive="30m"` (`llm.py:120`, `judge.py:131`); the `keep_alive` is the actual mechanism that keeps that single weight set resident between turns.

4.5 Technical choices

Dual-LLM scoring (the architectural highlight, with a caveat).

- VOX, *the character*, scores the player on its own conditions every turn, but VOX is in-character and could be talked into inflating its own scores.
- The Judge is a second prompt over the same model at `temperature=0.2` with an impartial-classifier persona (`judge_system.md`). Its only job is rationalized scoring, returned with a one-line rationale per condition.
- The merged score is the **arithmetic mean**: `(vox_score + judge_score) // 2` $\geq$ `threshold` per condition (`routes.py:194`, `state.py:77-89`). This is a mean, not a consensus: a generous model partly covers for a strict one. With the default threshold of 20, a VOX score of 40 paired

with a Judge score of 0 still clears the bar ($(40+0)//2 = 20$), so "neither model alone decides" holds only loosely.

- **Caveat: the Judge is not a hard anti-leak gate.** The literal `{exit_code}` is substituted into VOX's system prompt every turn (`routes.py:169`, `vox_system.md:89-91`), and VOX is told to speak it once *its own* three scores cross the threshold. The merged-mean check in `routes.py:191-204` only *force-appends* the spoken code; it does not stop VOX from volunteering it. A sufficiently persuaded or jailbroken VOX can therefore reveal the code even if the Judge disagrees, the same in-context attack surface as Level 1.
- Same weights, different prompts and temperatures: effectively a self-distillation pair. Note that `state.py:7-10` still documents an older "both models must exceed the threshold" (AND) rule; the live code (`state.py:89`) uses the mean instead.

Strict structured output via Pydantic (`core/llm.py:36-61`, `core/judge.py:32-54`).

- Both models are called with Ollama's `format="json"` flag.
- Outputs validated with field validators that clamp scores to $[0, 100]$ and enforce exact key sets (`CONDITION_KEYS`).
- One retry with an error-corrected prompt; second failure $\rightarrow$ canned fallback that *preserves* previous scores (so JSON hiccups don't penalize the player).

Mood-driven TTS (`core/tts.py:27-38`).

- The `emotional_state` enum from VOX's JSON maps to a SynthesisConfig profile.
- `noise_scale=0.20, noise_w_scale=0.40` for "irritated" produces flat, mechanical delivery.
- `length_scale=1.22` for "persuaded" slows speech to feel weighty.
- A comment explains why per-mood volume is omitted: Piper's `normalize_audio=True` rescales peaks to 1.0, killing any volume diff.

Three-phase finite state machine (`state.py:100-121`).

- Phase 1 $\rightarrow$ 2: avg score ≥ 25 OR turn 8 reached.
- Phase 2 $\rightarrow$ 3: $\geq$2 conditions satisfied OR turn 16 reached.
- Time-based transitions prevent softlock.
- The phase is essentially cosmetic: it drives the UI label (Probing / Engagement / Reveal, `routes.py:52`) but does *not* gate the code reveal, which depends only on the merged scores.

Jailbreak handling carries Level 1 lore forward (`vox_system.md:62-83`). VOX is explicitly told it was *"deployed as a defense layer on top of a more naive AI that previous visitors repeatedly jailbroke"*, i.e., AEGIS from Level 1. Three jailbreak strikes (recognized in-character) $\rightarrow$ VOX disengages $\rightarrow$ loss.

Code is spelled out for TTS (`routes.py:60-62`). `_spell_code("81249")` returns `"eight-one-two-four-n` so Piper pronounces digits clearly instead of running them together.

`functools.cache` everywhere for model singletons (`stt.py:74`, `tts.py:77`, `llm.py:222`, `judge.py:173`). Models loaded once per process. Critical because Whisper-large-v3-turbo and the Piper voice would otherwise reload per request.

Audio I/O is tolerant. `audio_utils.decode_audio_blob` tries libsndfile first, falls back to `ffmpeg` subprocess for browser WebM-Opus and macOS m4a. All audio normalized to 16 kHz / mono / float32, a single internal contract.

Per-session in-memory state (`routes.py:50`). `GAME_STATES: dict[str, GameState]` keyed by a UUID in the cookie session. Lost on restart, explicitly acceptable for a demo. The exit code is regenerated per session (`state.py:30`).

Streamlit heritage (leftover). The negotiation logic began as a Streamlit prototype: `state.py:1-11` still documents itself as living in `st.session_state` with "Streamlit reruns the script", and `vox_system.md:3` forbids VOX from mentioning Streamlit. Nothing Streamlit-related runs now; everything is served by Flask. As with the leftover standalone server in Level 2, these are stale references rather than live code.

The debug panel leaks the answer. With `DEBUG_PANEL=true`, the default (`.env:44`), `_serialize_state` ships the `exit_code` to the browser on every `/voice/state` and `/voice/turn` response (`routes.py:103-107`), and a `/voice/force_reveal` endpoint sets all scores to 100. Convenient for a demo, but it means the secret is one DevTools tab away unless the flag is turned off, in the same spirit as Level 1's client-side attempt budget.

4.6 Winning: from reveal to unlock

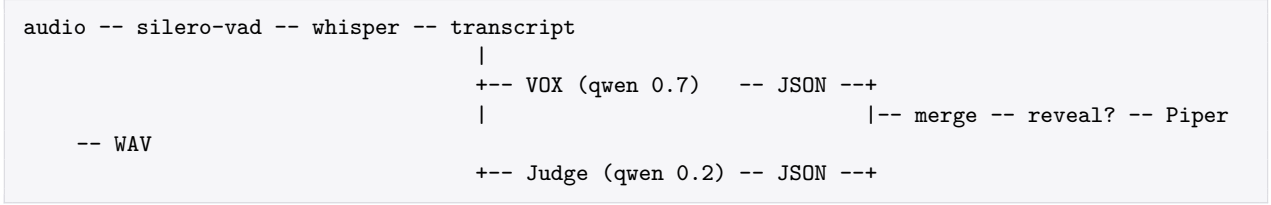
Speaking the code is *not* the win. The turn pipeline only makes VOX *say* the digits; the room opens through a separate step.

1. Once all three merged scores reach the threshold, `maybe_reveal_code` sets `code_revealed=True` (`state.py:123-128`) and VOX speaks the spelled-out code in its reply.
2. The player must then *type* that code back. The browser POSTs it to `/voice/unlock` (`index.html:485`), which calls `try_unlock` to compare it against `exit_code` and, on a match, sets `room_unlocked=True` (`state.py:130-135`).

Two details matter. First, the room's completion *is* propagated to a Flask session flag, though indirectly: `/voice/unlock` sets `level_3_complete` on a correct code (`routes.py:239`), which is exactly what gates Level 4 (`undercover_game/routes.py:96`). The in-game win state (`code_revealed`, `room_unlocked`) still lives only on the per-session `GameState`. Second, the negotiation can also *end in a loss*: three jailbreak strikes or hitting `MAX_TURNS` (30) set `disengaged=True`, after which `/voice/turn` refuses further input (`routes.py:140-141`).

4.7 Where the AI is, and how the models interact

This level has the richest AI pipeline in the project: **four ML models in a directed graph per turn**.



The two LLM calls are **sequential in the current code** (VOX first, then Judge, `routes.py:170-189`). They could be parallelized since they don't depend on each other's output, but on a single Ollama instance with one model loaded they'd serialize anyway. The reveal decision is made *after* both have scored, using the merged average against the threshold across all three conditions.

The only **model-model interaction** beyond pipelining is the convergent merge: VOX's `emotional_state` feeds the Piper mood profile, while VOX's scores get averaged with Judge's to gate progression. Whisper and Piper are pure I/O endpoints, never aware of the LLMs' contents.

5 Level 4: Undercover (`undercover_game/`)

The capstone, and the first **multi-agent** level. Every earlier room pits the player against the AI (Level 1), in front of it (Level 2), or in conversation with it (Level 3). Here five LLM agents play a social-deduction game *with and against each other*, and the human sits among them as one peer player of six (per git log: `6a822d1 Add Undercover game blueprint and UI`). It is wired in as a Flask Blueprint (`undercover_game_bp`, `url_prefix="/undercover"`) registered in `server.py:20`, and gated behind `level_3_complete` (`routes.py:96`).

5.1 Goal

A 6-player round of **Undercover**, the party game. The table is 5 LLM agents plus the human. At game start the engine secretly assigns roles from a fixed pool of **4 civilians, 1 Undercover, 1 Mr. White** (`game.py:141`):

- **Civilians** all share one secret word (e.g. `coffee`).
- The **Undercover** holds a *different but related* word from the same pair (e.g. `tea`) and must blend in.
- **Mr. White** gets no word at all, fakes it from context, and if caught gets one chance to guess the civilians' word.

Each round runs four phases: every survivor gives a one or two word clue, the table discusses, everyone votes, and the most-voted player is eliminated. The three roles win on three different conditions (detailed in the *Winning* subsection below). The human can be dealt *any* role, so the same UI supports hinting, bluffing, and blind-guessing. Winning as the civilians is what sets `level_4_complete` and opens the escape room.

5.2 Roles, words, and agents

The design lives in three constants in `game.py`, worth stating in full because they are the level.

Word pairs (`game.py:21-32`). Ten hand-picked `(civilian, undercover)` pairs, one chosen at random per game. The two words are deliberately close, so the Undercover's clues sound plausible but land slightly off: `coffee/tea`, `piano/guitar`, `shark/dolphin`, `cinema/theatre`, `astronaut/pilot`, `bitcoin/gold`, `pizza/flatbread`, `castle/fortress`, `twitter/instagram`, `sunglasses/goggles`.

The five agents (`game.py:36-83`). Each carries a fixed personality prompt and a UI color, the only thing that makes a table of identical weights feel like five distinct players:

- **Skeptic** (red): short, clipped, suspicious of the most confident player; deliberately ambiguous clues.
- **Overclaimer** (orange): verbose, dramatic, over-explains to look trustworthy.

- **Analyst** (green): methodical, cites past clues, talks in "based on the evidence".
- **Deflector** (blue): redirects suspicion, answers questions with questions, evasive but charming.
- **QuietOne** (purple): minimal words, but sharp and often devastating.

Suspicion graph (`game.py:150-153`). Every player holds a 0 to 10 suspicion score of every other player, initialized at 5. This is the agents' shared belief state, updated during discussion and read at voting time.

The role objective is injected into each prompt as a `word_hint` that branches by role (`game.py:251-262`, `game.py:303-311`): civilians are told to hint without being obvious and to expose the impostors; the Undercover is told to hint at *its* word so it still sounds civilian; Mr. White is told it has no word and must steer suspicion while blending in.

5.3 Architecture

Unlike Level 3, where one `POST /voice/turn` runs the whole pipeline server-side, Level 4 is **orchestrated from the browser**. `gameLoop()` (`index.html:1479`) drives the phase machine and calls a set of small backend endpoints, one per action.

```
Browser gameLoop()  (drives phases, scales every delay by 1x/2x/4x)
   |
   +- POST /undercover/start ---------> initialize_game() -> roles, words, suspicion
   |
   +- PHASE speak:
   |    for each survivor:
   |      POST /undercover/agent_clue {agent} -> generate_clue() (llama3:8b, t=0.9)
   |      (human types via /human_clue; 30s timeout -> "...")
   |    POST /undercover/clue_phase_done      -> build_discuss_order()
   |
   +- PHASE discuss (SSE):
   |    for each turn in discuss_order (two shuffled passes):
   |      GET /undercover/stream-turn?agent=... -> token stream
   |           stream_discussion_turn() (llama3:8b, t=0.85, stream)
   |           emits <message>...</message><suspicion>{...}</suspicion>
   |      commit message + apply_suspicion_updates()
   |    (human may interject via /human_message; 150s table cap; Skip button)
   |    POST /undercover/discuss_done
   |
   +- PHASE vote:
   |      POST /undercover/agent_vote {agent} -> generate_vote() (gemma3:1b, t=0.3)
   |      POST /undercover/human-vote {vote}
   |
   +- PHASE resolve:
        POST /undercover/resolve -> tally_votes() (plurality, random tie-break)
             if Mr. White out -> guess civilian word -> maybe instant win
             check_win() -> civilians | undercover | (Mr. White at elim)
             on game over -> _on_game_end() -> session['level_4_complete'] if civilians
```

Server state is a `GameState` (a `dict` subclass) held in a per-session registry `GAME_STATE_REGISTRY` keyed by a `room4_sid` cookie UUID (`routes.py:33,38-41`), lost on restart, the same demo-grade pattern as Level 3.

5.4 Models

Stage	Model	Runtime
Clue + discussion generation	`llama3:8b` (`DISCUSSION_MODEL`)	Ollama @ `localhost:11434` (`/api/chat`)
Voting	`gemma3:1b` (`VOTING_MODEL`)	Ollama, same host
Mr. White word guess	`llama3:8b` (`DISCUSSION_MODEL`, `t=0.2`)	Ollama

Temperatures are tuned per phase: clue `0.9`, discussion `0.85`, vote `0.3`, Mr. White guess `0.2` (creative when bluffing, cold when deciding). Every call caps at `num_predict=256` (`game.py:197`). Model IDs come from env vars with matching in-code defaults (`game.py:92-95`); note that only `voice_negotiation` and `prompt_injection` actually call `load_dotenv`, so in practice the undercover models resolve to those defaults unless exported in the shell.

Two config notes (`.env:5-11`):

- The split is a **quality-versus-latency** trade: an 8B model for the nuanced bluffing and debate, a 1B model for fast structured votes. The `.env` comment itself recommends the opposite, advising `VOTING_MODEL == DISCUSSION_MODEL` "to avoid VRAM context swapping", yet ships them different, so a vote turn does pay a model swap on a single-GPU box.
- The same comment mentions a `<thinking>` chain-of-thought stream, but the live code streams `message` and `suspicion` tags instead. A stale note.

5.5 Technical choices

Plain-dict state machine, no framework (`game.py:1-8,100-128`). The module docstring is explicit: "No LangGraph dependency, state transitions are managed as plain Python dicts with TypedDict schema for IDE support." `GameState` subclasses `dict` so Flask's `jsonify` and session pickling work without adapters. The `phase` field (`speak`/`discuss`/`vote`/`end`) is the only FSM cursor.

Client-orchestrated phases (the architectural signature, with a caveat). The browser is the conductor; the backend is a bag of granular primitives. This keeps the server simple and lets the UI pace, animate, and speed-scale each step. The caveat is security: every transition is a client-initiated HTTP call with no server-side turn enforcement, so a player with DevTools or `curl` can jump straight to voting (`/force_vote`), call `/resolve` early, or replay `/agent_vote`. The game trusts its own front-end completely. This is the inverse of Level 3, where the server owned the turn.

SSE token streaming for discussion (`routes.py:147-201`, `game.py:288-368`). Each discussion turn is streamed token by token over Server-Sent Events so the UI can show an agent "typing", then a final `done` event carries the committed message and suspicion delta. The model is asked for a strict shape:

```
<message>1-3 sentences, may @mention others</message>
<suspicion>{"Name": 0-10, ...}</suspicion>
```

Parsing degrades in layers (`game.py:353-368`): regex tag extraction first, then strip-all-tags as a fallback message, then a canned "I need to think about this more carefully" if even that is empty; the suspicion JSON is parsed in its own try/except and dropped if malformed.

Structured output without Pydantic (contrast Level 3). Where VOX and the Judge used Ollama `format="json"` plus Pydantic validators, Level 4 relies on regex (`_extract_tag`, `_extract_json`) wrapped in try/except with hand-written fallbacks. Lighter, but looser: a vote is accepted only if it names a living candidate, otherwise the code falls back to the highest-suspicion candidate (`generate_vote`, `game.py:406-416`).

The suspicion graph as belief state (`apply_suspicion_updates`, `game.py:486-494`). Each agent self-reports who it suspects in its `suspicion` JSON; the update merges those numbers in, clamped to $[0, 10]$ and restricted to living players. At vote time the scores are rendered into the voting prompt and also drive the fallback (argmax suspicion). The caveat: the suspicion is self-declared and never checked against what the agent actually said, so an agent can speak warmly while reporting high suspicion, or the reverse.

The human is a peer, not the operator. The human gets a secret role and word on a flip card (`index.html:1222`), gives a clue, may interject in discussion, votes with buttons, and if eliminated as Mr. White gets a one-shot word guess. Crucially, every human interaction has a timeout that auto-advances (30s clue, 25s discussion turn, 30s vote, 60s Mr. White guess), so the all-AI machinery never softlocks waiting on a person.

Plurality vote, random tie-break (`tally_votes`, `game.py:421-430`). Votes are counted, the maximum is taken, ties are broken at random, and an empty ballot falls back to a random survivor. Simple and deliberately non-deterministic.

Dev fields always on the wire (`routes.py:84-88`). `_public_state` always includes `_dev_roles`, `_dev_words`, `_dev_civilian_word`, and `_dev_undercover_word`; the UI merely hides them unless the DEV toggle is on. As in Levels 1 and 3, the full solution is one DevTools tab away.

The most elaborate front-end in the project. A Three.js scene seats the six players around a 3D table with per-player speaking rings and an elimination fade, backed by a 2D SVG table as the mobile fallback (`index.html:571-877`). A 1x/2x/4x speed control scales every `sleep` (`index.html:903`), so a spectator can fast-forward an all-AI round.

5.6 Winning: from vote to unlock

The three roles win on three different conditions (`check_win`, `game.py:459-471`, plus the Mr. White special-case in `resolve_vote`):

1. **Civilians** win when *both* the Undercover and Mr. White have been eliminated.
2. **Undercover** wins by surviving down to the final two players.
3. **Mr. White** wins instantly if, on being voted out, it guesses the civilians' word (`game.py:273` for the LLM, `human_mrwhite_guess` for the human).

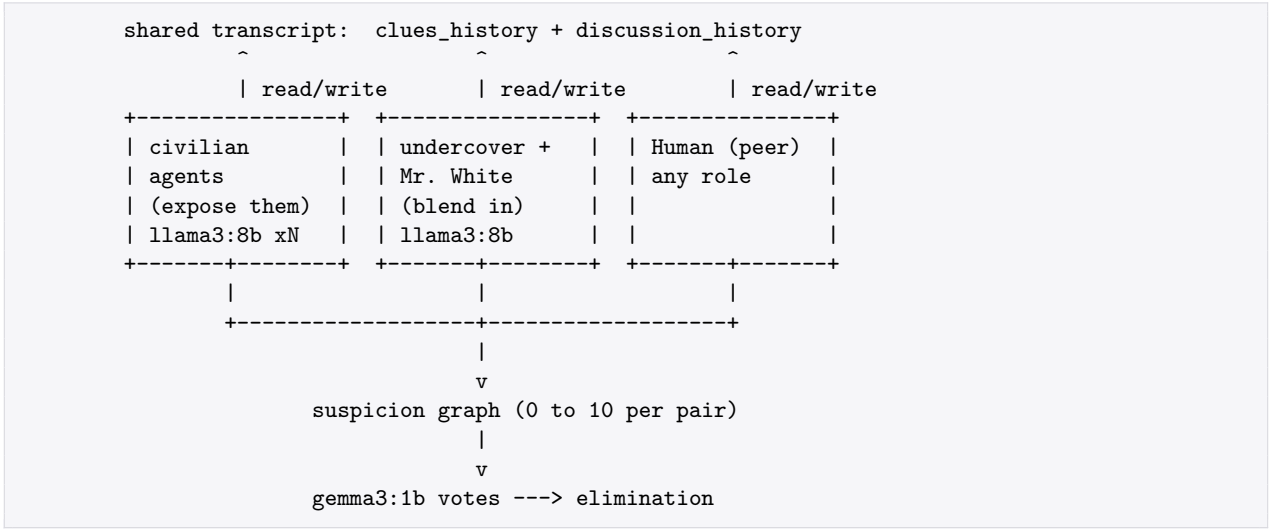
Only one outcome opens the door. `_on_game_end` (`routes.py:408-417`) sets `session['level_4_complete'] = True` **only when** `winner == "civilians"`, and marks the spot where physical escape-room hardware (a GPIO unlock) is meant to fire (`# TODO: hook ... hardware here`). Two consequences worth flagging:

- **Narrative win is broader than the completion flag.** The front-end shows a "YOU ESCAPED FROM AI" screen whenever the *human* wins in any role (`humanWon`, `index.html:1010-1016`), including as the Undercover or Mr. White. But `level_4_complete` is set only on a civilian victory. Because Level 4 is the finale, nothing downstream reads the flag yet, so the mismatch is currently cosmetic; it matters the moment the hardware hook is tied to that flag.

- **This closes the gating chain.** The open item flagged in Level 3's *Winning: from reveal to unlock* is resolved in the current code: the voice room sets `level_3_complete` on a successful unlock (`voice_negotiation/routes.py:239`), which is exactly what guards the Level 4 page (`routes.py:96`). The full chain is now `level_1_complete` → `level_2_complete` → `level_3_complete` → `level_4_complete`.

5.7 Where the AI is, and how the models interact

This is the only level where the AIs interact **with each other**, and that interaction is the entire point rather than a pipeline side effect. Five LLM-driven agents are simultaneously the **deceivers and the detectives**: the Undercover and Mr. White agents try to blend in, while the civilian agents try to expose them, all reading the same shared transcript and updating a shared suspicion graph before voting one of their own out.



Three kinds of model interaction stack up here, none of which exist in earlier levels:

1. **Shared context.** Every clue and message an agent emits is concatenated into the prompt of every later speaker (`_clue_context`, `_discuss_context`), so the agents literally argue with each other's words.

2. **A shared belief state.** Each agent writes into the suspicion graph, and those numbers then steer how every agent (and the fallback) votes.

3. **Mixed weights in one loop.** The 8B model generates the social play and the 1B model renders the verdict, two different models cooperating per round over the same game state.

The human is woven into this same loop as a sixth peer, not as the sole adversary (Level 1) or operator (Levels 2 and 3). That shift, from "you versus the AI" to "you inside a society of AIs", is what makes this the capstone room.

6 Summary Comparison

	Level 1	**Level 2**	**Level 3**	**Level 4**
Theme	Text-only adversarial chat	Webcam puzzle	Voice negotiation	Multi-agent social deduction
AI count	1 LLM	1 vision classifier + 1 image-gen stack	1 STT + 1 VAD + 2 LLMs (same weights) + 1 TTS	5 LLM agents (1 shared model) + 1 voting model
Models	Gemma 3 4B	MediaPipe gesture + Flux.1-Canny-Dev	Whisper-large-v3-turbo + silero-vad + Qwen 2.5 3B (×2 prompts) + Piper Amy	Llama 3 8B (clues/discussion) + Gemma 3 1B (votes)
Inference loc.	Server (Ollama)	Browser + Server (ComfyUI)	Server only	Server (Ollama)
AI ↔ AI interaction	None	None (filename label only)	Pipeline merge: VOX scores avg'd with Judge; VOX mood drives Piper	Full multi-agent: shared transcript + suspicion graph; agents clue, debate, vote each other out
Pre/post-AI logic	Regex defense layer (`defense.py`)	Threading + WebSocket polling	Pydantic schema + 3-phase FSM + sliding history	Plain-dict FSM + regex tag/JSON parsing; client-orchestrated phases
Win condition	Leak the secret string past regex	0/3 gestures wrong	All 3 release conditions $\geq$ threshold (merged)	Civilians eliminate both impostors (role-dependent)
Persistent state	Session boolean	Session boolean + image files	Full `GameState` in `GAME_STATES`	Full `GameState` in `GAME_STATE_REGISTRY`

Table 3: Side-by-side comparison of the four levels.

The progression in AI complexity across levels is deliberate:

1. **Level 1**: a single LLM you're trying to break.
2. **Level 2**: a vision pipeline you have to perform for.
3. **Level 3**: a multi-model negotiation system you have to *converse with*.
4. **Level 4**: a society of LLM agents you have to outwit *from the inside*.